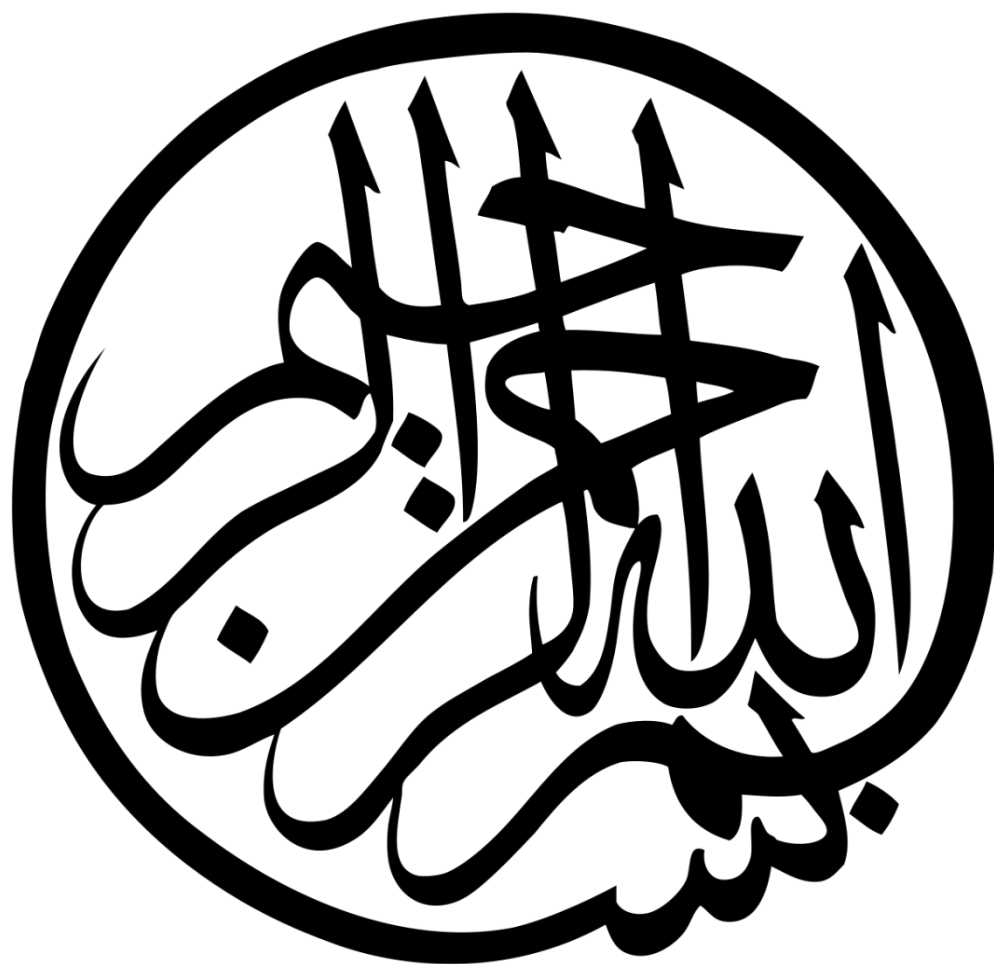




# گزارشکار پروژه کارشناسی

## عنوان : قفل تشخیص چهره

استاد راهنما : دکتر محمد حسن آسمانی

تهیه شده توسط دانشجو حسین داودی

با شماره دانشجویی 9632801

پاییز 1400

تقدیر و تشکر از استاد راهنمای همراه در مسیر، جناب آقای دکتر آسمانی به دلیل راهنمایی  
ها و مشاوره های تاثیرگذار ایشان .

## مقدمه:

ابتدا توضیحات کلی و مقدماتی راجع به کلیات این را باید ارائه کرد. این فعالیت دارای بخش های عملی سخت افزاری ، نرم افزاری و همچنین کد نویسی در دو محیط آردوینو و پایتون را شامل میشود. تعریف کلی پروژه در بستر iot (اینترنت اشیاء) میباشد و از سخت افزار های خاص این محیط نیز بهره برده شده است.

این گزارش تلاش میکند علاوه بر توضیح جزییات مختلف این فعالیت، رابطه و ارتباط آنها را نیز بیان کند.

## بخش اول: ایده کلی

براب ساختن قفلی که بوسیله تشخیص دادن چهره، باز میشود چه چیزهایی لازم داریم؟ موارد ابتدایی که به ذهن ما میرد اول داشتن یک قفل در وهله اول و سپس یک دوربین که چهره افراد را بتواند ببیند . حال باید به دنبال یک کانال ارتباطی بین قفل فیزیکی و تصویر مشاهده شده توسط دوربین باشیم! برای این ارتباط از ابزارهای بستر *iot* استفاده میکنیم. یعنی تصویر را بصورت داده بوسیله کدها به یکسری داده تبدیل میکنیم و حالا این داده با تصویری که از قبل تهیه شده و بعنوان فرد مورد قبول سیستم میشناسد مقایسه میکند و در صورت مقبول بودن و داشتن خطایی کمتر از مرز ما ، دستور باز شدن قفل را باید به سمت قفل بفرستد! فرستادن این دستور با استفاده از آردوینو و بصورت بیسیم میباشد که در قسمت های بعد با جزئیات گفته خواهد شد. حال که دستور به قطعه (*esp*) رسید با توجه به کد و چیزی که از قبل روی آن آپلود کردیم عمل خواهد کرد ، یعنی اکنون وارد مرحله فیزیکی و سخت افزاری شده ایم و بوسیله ابزاری که در بخش سخت افزار توضیح داده خواهد شد ولتاژ فعال کردن را به قفل اعمال میکنیم و زبانه قفل را دچار تغییر وضعیت میکنیم!

## بخش دوم: مدار فیزیکی

قطعات الکترونیکی مختلفی در این پروژه استفاده شده که اگر بخواهیم از آنها نام ببریم شامل: منابع مختلف ولتاژ ، میکروچیپ esp8266 ، رگلاتور ، رله ، قفل الکتریکی ، برد مورد و ... .

مرکز اصلی و قطعه اصلی کار ما چیپ esp8266 میباشد که مجهز به وایفای بوده مناسب برای ارتباطات بی سیم مانند شبکه lan یا اینترنتی میباشد. برای راه اندازی نیاز به ولتاژ 3.3 ولت دارد و پس از اینکه برنامه ریزی شد، یکی از پین های که میتواند تغییرات خروجی (High/Low) رویش اعمال شود را (که درون کد آردوینو مشخص است) را بعنوان عامل اجرا کننده دستور باز و بسته شدن قفل انتخاب کرده و سر یک سیم را بوسیله برد بود به این پین وصل میشود.

از طرفی دیگر باید رله را راه اندازی کرد به این صورت که از طریق منبع ولتاژ 12 ولت و عبور دادن ولتاژ از یک رگولاتور تنظیم کردن خروجی روی 5 ولت (ولتاژ مورد نیاز برای تامین رله)، رله راه اندازی شده. سپس پین خروجی esp را بوسیله سیمی که از قبل اشاره شد به تریگر رله وصل کرده و با تغییر وضعیت تریگر میتوان ولتاژی را که به پایه com وصل شده را از طریق یکی از خروجی های NO یا NC به قفل وصل کرد و سر دیگر قفل را به زمین متصل کرد. ولتاژ داده شده به پایه com با توجه به نوع قفل و ولتاژ مورد نیازش میباشد!

دلیل دخیل کردن رله در این مدار این است که جریان خروجی esp خیلی کم میباشد و نمیتواند روی قفل تأثیری بگذارد ، بخاطر همین از رله استفاده میکنیم تا جریان خروجی بیشتری (حدود 1 آمپر) را به قفل برای تغییر اعمال تغییر وضعیت ایجاد کند!

توجه شود که چون که ولتاژ تامین esp و رله از دو منبع متفاوت تامین شد، gnd همه قطعات را یکی میکنیم!

## بخش سوم: رابطه server/client در esp و کد آردوینو

در این ارتباط server/client لبتاب (دوربین تشخیص چهره) نقش server را ایفا میکند و چیپ esp در نقش client میباشد. میخواهیم بوسیله wifi موجود در esp و وافای لبتاب، یک ارتباط سروری وصل کنیم، به این صورت که به نقطه اتصال بوسیله موبایل یا یک روتر اینجاد کرده و لبتاب به آن وصل میشود. برای وصل کردن چیپ به نقطه اتصال از IDE آردوینو استفاده میکنیم و با اضافه کردن header های مربوط به بخش wifi به کد، داخل بخش setup اسم نقطه اتصال و رمز عبورش را معرفی که در و در پنجره سریال یک نشانگر برای اینکه بدانیم در حال وصل شدن است میگذاریم. یک پین رو به عنوان خروجی مشخص کرده و روی High میگذاریم چون رله ما با low شدن تریگر میشود. Ip لبتاب را معرفی کرده و پورت را نیز مشخص میکنیم (سرور و کلاینت هر دو باید این داده ها یکی باشد تا اتصال برقرار شود). در قسمت اصلی کد آردوینو ابتدا چک میکنیم که آیا به سرور وصل شده ایم یا نه در صورت وصل نشدن دوباره پس از مدتی چک میکنیم. همینطور ادامه میدهیم تا اینکه بالاخره وصل شده باشیم. در صورت وصل شدن، از قبل یک flag قرار داده بودیم که یکی از شرطهای تغییر وضعیت پین ما میباشد که اگر false باشد قفل همیشه بسته خواهد بود (منظور از بسته بودن قفل، high بودن پین خروجی مورد نظر است). برای اینکه این شرط بولین ما به حالت true در بیاید کلاینت باید پیام OPENM را ارسال کند وقتی ارسال کرد گیرنده بصورت کارکتر کارکتر این پیام را دریافت میکند و پشت سر هم به عنوان یک استرینگ قرار میدهد. کارکتر M بعنوان پایان درخواست ما میباشد و وقتی M دیده میشود چک میشود که آیا قبلش OPEN بوده یا نه و در صورت صحت این موضوع پین را Low کرده که در نهایت موجب باز شدن قفل میشود! در وهله بعد بولین را تغییر میدهیم تا یکی از شروط برگشت به حالت قفل بودن را تامین کنیم و شرط دیگر گذشتن زمان محدودی که خودمان تعیین کرده ایم میباشد که بوسیله تایمر خود چیپ مقدار زمان را میخوانیم و دو ثانیه بعد آنرا بوسیله قرار دادنش در شرط if که برای برگشتن به حالت قفل بودن بود، قفل را دوباره میندیم و پایه را high میکنیم. استفاده نکردن از delay برای قرار نگرفتن چیپ در حالت گیچی بود که بجای آن از تابع millis استفاده کردیم.

حال برای اینکه ابتدا قبل از زدن کد های پردازش تصویر و بوسیله یک پیام از سمت سرور قفل را باز بسته کنیم و ارتباط سرور و کلاینت را چک کنیم از برنامه socketTest استفاده میکنیم. به این صورت که پورت و ip لبتاب را در کد آردوینو قرار داده و معرفی میکنیم سپس در این برنامه هم همین کار را میکنیم و سرور را در این برنامه راه میاندازیم. میبینیم که با ارسال پیام OPENM از سمت سرور به کلاینت (چیپ) پین دچار تغییر وضعیت به مدت محدود میشود.

## بخش چهارم: کدهای python

این بخش از سه کد مجزا تشکیل داده شده که اولی و دومی برای دریافت و تحلیل داده هستند و سومین بخش که بخش اصلی میباشد برای ارتباط برقرار کردن و تشخیص چهره و اجازه باز شدن قفل کاربرد دارد.

### کد اول (label):

کار کلی این برنامه این است که قبل از هرکاری که مرتبط با باز کردن قفل میباشد، تصاویرهای چهره فرد مورد نظر را برای ما بگیرد یا به عبارتی دیگر داده را تهیه کند و عکس های چهره فرد را در پوشه ای ذخیره کند تا در قسمت بعد آنها را تحلیل و تجزیه کنیم.

سربرگهای مربوط به پردازش تصویر و ایجاد تغییرات در سیستم عامل را اضافه کرده و سپس از شبکه هوش مصنوعی `haarcascade_frontalface_default.xml` معرفی کرده که از آن برای تشخیص چهره افراد از یک درون یک تصویر استفاده میکنیم یعنی ابتدا از وبکم استفاده میکنیم و تصویر را در یک متغیر ذخیره میکنیم سپس تصویر رنگی را به حالت سیاه سفید در میآوریم (تا قابل استفاده در شبکه هوش مصنوعی باشد) و سپس از کد تشخیص صورت در عکسها استفاده میکنیم و سپس این تصاویر را در پوشه ذکر شده ذخیره میکنیم. حدودا 30 تصویر از چهره را با تاخیر کمی نسبت ذخیره میکنیم تا همه تصاویر کاملا مشابه به هم نشوند و بعدا در تشخیص چهره به مشکل نخوریم.

توجه شود که این بخش فقط برای تشخیصی صورت افراد در یک تصویر است و نه برای شناسایی چهره.

### کد دوم (Train):

این قسمت بطور خلاصه برای ذخیره داده ها و دسته بندی چهره های افراد مختلف (در صورت وجود) و train کردن آنها و جایگذاری این داده ها درون فایل `trainer.yml` پس از فهمیدن (train) تک تک عکسها میباشد.

سر برگهای مختلف شامل برای پردازش تصویر، ساختن آرایه، باز کردن فایل های تصویری و ذخیره دائمی را ابتدای کد خود قرار میدهم سپس دوباره شبکه هوش مصنوعی تشخیص صورت در تصویر را شامل میکنیم. همچنین یک وسیله (شیء) برای شناسایی چهره فرد قرار میدهم که داده ها بعدا بوسیله آن train میشوند. اکنون مسیری که فایل های تصویری از بخش قبل را ذخیره کردیم معرفی میکنیم و از label هایی که سری قبل استفاده کردیم برای تفکیک تصاویر مربوط به هر فرد (در صورت وجود) استفاده میشود. به این صورت که بوسیله دستور `os.walk` در آدرس محل قرار گیری تصاویر میرویم و فایل به فایل تصاویر را چک میکنیم، تصاویر در هر پوشه ای که باشند، به عنوان `root` و از اسم پوشه ای که تصاویر درونش قرار دارند label میخورد که به هر label شماره ای تعلق گرفته است. حالا هر تصویر لود میشود و یک آرایه تشکیل میدهد، دوباره از تشخیص صورت در عمس استفاده میکنیم (برای



افزایش دقت) . سپس در ناحیه تصویر صورت داده مربوط عکس (roi) را در آرایه xTrain اضافه میکنیم و عدد مربوط به label را نیز هم به عنوان راهنمای داده در آرایه yLabels قرار میدهیم. پس از اینکه برای تک تک عکسها این عملیات را انجام دادیم، label ها را از آنجا که مربوط به افراد هستند و خاص هستند در یک متغیر دایم ذخیره کرده و در نهایت داده های آرایه های xTrain و yLabel را بوسیله recognizer آموزش میدهیم و در فایل trainer.yml ذخیره میکنیم.

## کد سوم (use):

این کدی است که برای فعال بودن و کارکرد قفل همیشه باید در حال اجرا و آنلاین باشد که متشکل از دو بخش میباشد. بخش اول ساختن یک سرور برای وصل شده esp و لبتاب به یکدیگر و بخش دوم فعال کردن دوربین و ثبت کردن چهره فرد جلوی دوربین و چک کردن اینکه آیا باید قفل را باز کند یا نه و رابط کاربری بین سیستم و کاربر ایجاد کند.

دوباره مثل قبل کتابخانه های لازم را که شامل پردازش تصویر ، ذخیره دائم ، ساخت رابط سوکت و threading ( برای انجام دو فعالیت همزمان) اضافه میکنیم.

تمامی label ها و id های مربوط به آنها را با استفاده از pickle میخوانیم و در متغیری ذخیره میکنیم.

تابع socketServerRun را بصورت thread صدا میزنیم و سرور s را که با استفاده از socket() و دادن ip سیستم و پورتی که در آردوینو مشخص کردیم روی حالا listen قرار میدهیم که سرور باشد. این تابع یک تا همیشه تکرار میشود تا بالاخره به یک کلاینت وصل شود و یک آدرس و یک اسمی از کلاینت میگیرد و همینطور ادامه میابد. چون که ما فقط یک کلاینت در کارمان داریم بقیه این روند یافت کلاینت بر ایمان بی اهمیت میشود. در ادامه دوباره شبکه هوش مصنوعی برای شناسایی صورت در تصویر (faceCascade) و وسیله (شیء) شناسایی افراد را (recognizer) شامل میکنیم و داده های train شده در قسمت قبلی را میخوانیم.

قسمت پیش رو شروع کار بین کاربر و سیستم میباشد که از قبل دوربین را فعال کرده و در یک حلقه بینهایت با آوردن یک تصویر از کاربر میخواهد با فشار دادن یک دکمه کار را آغاز کند. پس از درخواست شروع از کار دوربین فعال میشود و دوباره از کاربر میخواهد در زاویه مناسب قرار بگیر و دکمه enter که معادل 13 میباشد را فشار دهد تا از حلقه داخلی break کرده و با فراخوانی تابع showFace شروع به عملیات تشخیص کند.

تابع showFace؛ بصورت مکرر شروع میکند و تصویر را از دوربین میخواند و بصورت سیاه سفید در می آورد سپس چهره فرد را از تصویر گرفته شده در جدا میکند و درون یک حلقه for با استفاده از recognizer داده این عکس جدید را با داده های train شده تست میکند و دو عدد به ما میدهد که یکی نزدیک بودن یک تصویر فلان فرد را نشان میدهد و دیگری اینکه فردی که تصویر دوربین شبیهش میباشد با چه عددی ذخیره شده؟ ، با گرفتن این عدد و استفاده از دیکشنری label که بوسیله pickle خواندیم میتوان label مربوط به عدد داده شده توسط recognizer را پیدا کرد که همان اسم فرد میتواند باشد ( با توجه به فرمت ذخیره داده اولیه در بخش های قبلی). حالا اگر عدد مربوط به نزدیک بودن تصویر کمتر از 48 باشد برای ما قابل قبول میشود و با کشیدن یک مستطیل اطراف چهره و نوشتن بخشی از مشخصات فرد تشخیص داده شده کنار مستطیل ، از کاربر خواسته میشود تا با فشار دادن هر دکمه اجازه وارد شده به حلقه ارسال داده پیام OPENM به تمامی کلاینت های متصل به سرور را بدهد. با ارسال این پیام به esp بقیه روند همانطور که قبلا گفته شد ادامه میابد و قفل را باز میکند. و اگر عدد شباهت بیشتر از 48 بود دوباره از اول همین تابع شروع به کار کند. وقتی قفل را باز کرد، برمیگردد به حلقه ای که showFace در آن صدا زده شده بود و دوباره آن حلقه را اجرا میکند.

پایان